

1 Config Generator

The generator creates ROS2-Monitor-Infra runtime YAML from a JSON generation request. The request is written as a deployment model: physical **hosts** contain executable **runtimes**, and **links** describe how hosts exchange records, DSL records, or verdicts.

1. discover visible ROS2 topics, services, and actions;
2. place runtimes on physical hosts;
3. select monitor, converter, and verdict service classes;
4. describe host links and transport kinds;
5. project the JSON request into one or more runtime YAML files.

2 Generation Request JSON

The JSON request has two top-level sections:

Section	Meaning
hosts	Physical devices. Each host owns the runtimes that execute on that device.
links	Connections between hosts. Each link names its from_host , to_host , endpoint runtimes, payload, and transport.

2.1 Runtime Kinds

Every runtime belongs to exactly one host and uses a **kind** field to select its shape.

kind	Meaning
ros2	The ROS2 graph visible on the host. Its sources are the monitorable resources. Source.source_kind is an enumeration: topic , service , or action .
monitor	Subscribes to selected ROS2 sources; outgoing DataRecords are routed by links.
converter	Reads records from input_from and writes DSL records to output_to . A converter may be a data_filter or a dsl_converter ; both are custom class-path based converter runtimes.
verdict_service	Reads DSL records or verdict records from input_from and sends verdicts to output_to .

Transport is not a standalone top-level section in the request. It is the implementation object inside a **Link**. Existing transport kinds are **file** and **mqtt**.

2.2 Abstract Schema

```
{
  "hosts": [
    {
      "id": "<host id>",
      "runtimes": [
        {
          "id": "<runtime id>",
          "kind": "ros2",
```

```

    "sources": [
      {
        "id": "<source id>",
        "source_kind": "topic | service | action",
        "name": "<ROS2 name>",
        "interface": "<ROS2 interface>"
      }
    ],
    {
      "id": "<runtime id>",
      "kind": "monitor",
      "subscribe": ["<source id>"]
    },
    {
      "id": "<runtime id>",
      "kind": "converter",
      "converter": "data_filter | dsl_converter",
      "class_path": "<module.path:ClassName>",
      "input_from": ["<source id | record id>"],
      "output_to": "<record id>"
    },
    {
      "id": "<runtime id>",
      "kind": "verdict_service",
      "class_path": "<module.path:ClassName>",
      "input_from": ["<record id>"],
      "output_to": [
        {
          "kind": "stdout | file | mqtt"
        }
      ]
    }
  ],
  }
],
"links": [
  {
    "id": "<link id>",
    "from_host": "<host id>",
    "to_host": "<host id>",
    "from_runtime": "<runtime id>",
    "to_runtime": "<runtime id>",
    "payload": "records | dsl | verdicts",
    "transport": {
      "kind": "file | mqtt"
    }
  }
]
}

```

3 Config Generation Algorithm

Algorithm 1 Generate runtime YAML from JSON

Require: generation request J

Ensure: generated YAML files F

```
1:  $F \leftarrow \emptyset$ 
2:  $Runtimes \leftarrow \text{INDEXRUNTIMES}(J.\text{hosts})$ 
3: for all host  $H$  in  $J.\text{hosts}$  do
4:   for all runtime  $R$  in  $H.\text{runtimes}$  do
5:     if  $R.\text{kind} = \text{ros2}$  then
6:       add  $R.\text{sources}$  to the source catalogue
7:     else if  $R.\text{kind} = \text{monitor}$  then
8:        $Y \leftarrow \text{NEWYAML}(H, R)$ 
9:       for all source id  $s$  in  $R.\text{subscribe}$  do
10:        add  $\text{MAKEMONITORED SOURCEENTRY}(s, J)$  to  $Y$ 
11:      end for
12:      for all outgoing link  $e$  from  $R$  where  $e.\text{payload} = \text{records}$  do
13:         $\text{records} \leftarrow \text{RESOLVELINKRECORDS}(e, J)$ 
14:         $\text{ATTACHDATARECORDEXPORTER}(Y, e.\text{transport}, \text{records})$ 
15:      end for
16:       $F[\text{YAMLFIL}(H, R)] \leftarrow Y$ 
17:     else if  $R.\text{kind} = \text{converter}$  then
18:        $Y \leftarrow \text{NEWYAML}(H, R)$ 
19:        $\text{chain} \leftarrow \text{MAKECONVERTERENTRY}(R)$ 
20:        $\text{chain}.\text{input\_from} \leftarrow \text{RESOLVEINPUTS}(R.\text{input\_from}, J)$ 
21:       for all incoming link  $e$  to  $R$  do
22:         $Y.\text{verdict\_runner}.\text{source} \leftarrow \text{MAKERUNNERSOURCE}(e.\text{transport}, e.\text{payload})$ 
23:       end for
24:       for all outgoing link  $e$  from  $R$  where  $e.\text{payload} = \text{dsl}$  do
25:         $\text{chain}.\text{dsl\_transport} \leftarrow e.\text{transport}$ 
26:       end for
27:       add  $\text{chain}$  to  $Y.\text{converters}$ 
28:        $F[\text{YAMLFIL}(H, R)] \leftarrow Y$ 
29:     else if  $R.\text{kind} = \text{verdict\_service}$  then
30:        $Y \leftarrow \text{NEWYAML}(H, R)$ 
31:       for all incoming link  $e$  to  $R$  do
32:         $Y.\text{verdict\_runner}.\text{source} \leftarrow \text{MAKERUNNERSOURCE}(e.\text{transport}, e.\text{payload})$ 
33:       end for
34:       add  $\text{MAKEVERDICTENTRY}(R)$  to  $Y.\text{converters}$ 
35:        $F[\text{YAMLFIL}(H, R)] \leftarrow Y$ 
36:     end if
37:   end for
38: end for
39:  $\text{WRITEYAMLFIL}(F)$ 
40: return  $F$ 
```

4 Runtime Resource Scanning

Algorithm 2 Scan ROS2 sources

Ensure: source catalogue G

```
1:  $T \leftarrow \text{run ros2 topic list -t}$ 
2:  $V \leftarrow \text{run ros2 service list -t}$ 
3:  $A \leftarrow \text{run ros2 action list -t}$ 
4:  $G \leftarrow \emptyset$ 
5: for all topic  $(name, interface)$  in  $T$  do
6:   add source  $(topic, name, interface)$  to  $G$ 
7: end for
8: for all service  $(name, interface)$  in  $V$  do
9:   add source  $(service, name, interface)$  to  $G$ 
10: end for
11: for all action  $(name, interface)$  in  $A$  do
12:   add source  $(action, name, interface)$  to  $G$ 
13: end for
14: return sorted  $G$ 
```

5 Projection Rules

5.1 Host and Runtime

Each `Host` becomes a placement boundary. The generator does not emit a YAML file for the host itself; it derives one YAML file for each executable runtime.

- `ROS2` runtimes provide `Source` definitions.
- `Monitor` runtimes capture sources and emit `DataRecords`.
- `Converter` runtimes build converter entries. `data_filter` and `dsl_converter` use the same class-path mechanism.
- `VerdictService` runtimes build verdict entries and output targets.

5.2 Source

Sources live under a `ROS2` runtime. A source is selected by id and then projected to the generated monitor YAML according to its `source_kind`:

- `topic` sources become entries under `topics`;
- `service` sources become entries under `services`;
- `action` sources become entries under `actions`.

5.3 Link and Transport

A `Link` connects two hosts. Its `from_runtime` and `to_runtime` fields name the endpoint runtimes. Its `transport` block is copied to the generated runtime YAML according to `payload`:

- `records`: the source side becomes a `DataRecord` exporter and the target side becomes a `verdict_runner.source`.
- `dsl`: the converter side becomes `dsl_transport`; the verdict side uses the same transport.
- `verdicts`: the source side becomes a verdict exporter and the target side becomes a `verdict_runner.source`.

The link does not repeat record ids. The generator derives the transported record set from the source runtime's `subscribe` or `output_to` field and the target runtime's `input_from`.

6 Concrete Example

JSON request:

```
{
  "hosts": [
    {
      "id": "robot",
      "runtimes": [
        {
          "id": "ros2_robot",
          "kind": "ros2",
          "sources": [
            {
              "id": "odom", "source_kind": "topic", "name": "/odom",
              "interface": "nav_msgs/msg/Odometry"
            }
          ]
        },
        {
          "id": "monitor_robot",
          "kind": "monitor",
          "subscribe": ["odom"]
        }
      ]
    },
    {
      "id": "converter_host",
      "runtimes": [
        {
          "id": "odom_speed_converter",
          "kind": "converter",
          "converter": "dsl_converter",
          "class_path": "custom.odom_speed_converter:OdomSpeedConverter",
          "input_from": ["odom"],
          "output_to": "odom_speed_record"
        }
      ]
    },
    {
      "id": "verdict_host",
      "runtimes": [
        {
          "id": "speed_verdict",
          "kind": "verdict_service",
          "class_path": "custom.odom_speed_verdict:OdomSpeedVerdict",
          "input_from": ["odom_speed_record"],
          "output_to": [{"kind": "stdout"}]
        }
      ]
    }
  ],
  "links": [
    {
      "id": "raw_records",
```

```

    "from_host": "robot",
    "to_host": "converter_host",
    "from_runtime": "monitor_robot",
    "to_runtime": "odom_speed_converter",
    "payload": "records",
    "transport": {"kind": "mqtt"}
  },
  {
    "id": "dsl_records",
    "from_host": "converter_host",
    "to_host": "verdict_host",
    "from_runtime": "odom_speed_converter",
    "to_runtime": "speed_verdict",
    "payload": "dsl",
    "transport": {"kind": "mqtt"}
  }
]
}

```